SIMATS ENGINEERING

CONTINOUS INTERNAL ASSESSMENT- ANALYTICAL PROBLEM SOLVING

COURSE NAME: DATABASE MANAGEMENT SYSTEMS
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Questions:05 **Total marks:100**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Afrose A (1924260309) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Afrose A

ANNEXURE
ANALYTICAL PROBLEM SOLVING

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	An organization maintains EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name, Location) relations. Using the given relations, apply selection (σ), projection (π), union (∪), set difference (−), Cartesian product (×), and join (⋈) operations to retrieve employees belonging to a particular department, department details, and employee–department information.	BL3 – Apply
2	A university maintains STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables. Write SQL CREATE TABLE statements with suitable PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. Ensure that referential integrity is maintained between the two tables.	BL3 – Apply
3	A retail company maintains a SALES(Sale_ID, Product_ID, Customer_ID, Quantity, Unit_Price, Sale_Date) relation. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions such as COUNT, SUM, AVG, MAX, and MIN to generate product-wise sales summaries, identify high-value products, and retrieve groups satisfying specified conditions.	BL3 – Apply
4	A company maintains EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID) records. Write a correlated subquery to identify employees whose salary is greater than the average salary of other employees working in the same department. Explain how the inner query is related to the outer query for each employee.	BL3 – Apply
5	A software company maintains EMPLOYEE(Emp_ID, Emp_Name, Dept_ID) and PROJECT(Project_ID, Project_Name, Emp_ID) relations. Apply INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, and CROSS JOIN to retrieve employee–project information. Explain the type of result produced by each join operation.	BL3 – Apply

DBMS Assignment – Relational Algebra with SQL Queries

Question:1

An organization maintains EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name, Location) relations. Using the given relations, apply selection (σ), projection (π), union (∪), set difference (−), Cartesian product (×), and join (⋈) operations to retrieve employees belonging to a particular department, department details, and employee–department information.

Given Relations:

```
EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID)
DEPARTMENT(Dept_ID, Dept_Name, Location)
```

1. Selection (σ)

Retrieve employees belonging to department 10.

```
SELECT * FROM EMPLOYEE
WHERE Dept_ID = 10;
```

Relational algebra: $\sigma_{Dept_ID=10}(EMPLOYEE)$

2. Projection (π)

Display department names and locations.

```
SELECT Dept_Name, Location
FROM DEPARTMENT;
```

Relational algebra: $\pi_{Dept_Name,Location}(DEPARTMENT)$

3. Union (∪)

Combine employee IDs from two compatible relations.

```
SELECT Emp_ID FROM EMPLOYEE1
UNION
SELECT Emp_ID FROM EMPLOYEE2;
```

Relational algebra: $E_1 \cup E_2$

4. Set Difference (−)

Find employee IDs in `EMPLOYEE1` but not in `EMPLOYEE2`.

```
SELECT Emp_ID FROM EMPLOYEE1
WHERE Emp_ID NOT IN (
   SELECT Emp_ID FROM EMPLOYEE2
);
```

Relational algebra: $E_1 - E_2$

5. Cartesian Product (×)

Combine every employee with every department.

```
SELECT *
FROM EMPLOYEE
CROSS JOIN DEPARTMENT;
```

Relational algebra: $EMPLOYEE \times DEPARTMENT$

6. Join (⋈)

Retrieve employee names with their department names.

```
SELECT E.Emp_ID, E.Emp_Name, D.Dept_Name
FROM EMPLOYEE E
JOIN DEPARTMENT D
ON E.Dept_ID = D.Dept_ID;
```

```
mysql> CREATE DATABASE CompanyDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE CompanyDB;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT (
    ->     Dept_ID INT PRIMARY KEY,
    ->     Dept_Name VARCHAR(50),
    ->     Location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TABLE EMPLOYEE (
    ->     Emp_ID INT PRIMARY KEY,
    ->     Emp_Name VARCHAR(50),
    ->     Salary DECIMAL(10,2),
    ->     Dept_ID INT,
    ->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO DEPARTMENT VALUES
    -> (10, 'IT', 'Chennai'),
    -> (20, 'HR', 'Bangalore');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO EMPLOYEE VALUES
    -> (1, 'Arun', 30000, 10),
    -> (2, 'Priya', 40000, 10),
    -> (3, 'Ravi', 25000, 20);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM EMPLOYEE
    -> WHERE Dept_ID = 10;
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|      1 | Arun     | 30000.00 |      10 |
|      2 | Priya    | 40000.00 |      10 |
+--------+----------+----------+---------+
2 rows in set (0.01 sec)

mysql> SELECT Dept_Name, Location
    -> FROM DEPARTMENT;
+-----------+-----------+
| Dept_Name | Location  |
+-----------+-----------+
| IT        | Chennai   |
| HR        | Bangalore |
+-----------+-----------+
2 rows in set (0.00 sec)

mysql> SELECT Emp_ID FROM EMPLOYEE WHERE Dept_ID = 10
    -> UNION
    -> SELECT Emp_ID FROM EMPLOYEE WHERE Dept_ID = 20;
+--------+
| Emp_ID |
+--------+
|      1 |
|      2 |
|      3 |
+--------+
3 rows in set (0.01 sec)
```

```
mysql> SELECT Emp_ID FROM EMPLOYEE
    -> WHERE Dept_ID = 10
    -> AND Emp_ID NOT IN (
    ->     SELECT Emp_ID FROM EMPLOYEE
    ->     WHERE Salary > 35000
    -> );
+--------+
| Emp_ID |
+--------+
|      1 |
+--------+
1 row in set (0.01 sec)

mysql> SELECT * FROM EMPLOYEE
    -> CROSS JOIN DEPARTMENT;
+--------+----------+----------+---------+---------+-----------+-----------+
| Emp_ID | Emp_Name | Salary   | Dept_ID | Dept_ID | Dept_Name | Location  |
+--------+----------+----------+---------+---------+-----------+-----------+
|      1 | Arun     | 30000.00 |      10 |      20 | HR        | Bangalore |
|      1 | Arun     | 30000.00 |      10 |      10 | IT        | Chennai   |
|      2 | Priya    | 40000.00 |      10 |      20 | HR        | Bangalore |
|      2 | Priya    | 40000.00 |      10 |      10 | IT        | Chennai   |
|      3 | Ravi     | 25000.00 |      20 |      20 | HR        | Bangalore |
|      3 | Ravi     | 25000.00 |      20 |      10 | IT        | Chennai   |
+--------+----------+----------+---------+---------+-----------+-----------+
6 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID, E.Emp_Name, D.Dept_Name, D.Location
    -> FROM EMPLOYEE E
    -> JOIN DEPARTMENT D
    -> ON E.Dept_ID = D.Dept_ID;
+--------+----------+-----------+-----------+
| Emp_ID | Emp_Name | Dept_Name | Location  |
+--------+----------+-----------+-----------+
|      1 | Arun     | IT        | Chennai   |
|      2 | Priya    | IT        | Chennai   |
|      3 | Ravi     | HR        | Bangalore |
+--------+----------+-----------+-----------+
3 rows in set (0.00 sec)

mysql>
```

Result: The six operations retrieve and combine employee and department information using relational algebra and SQL queries

Question:2

A university maintains STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables. Write SQL CREATE TABLE statements with suitable PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. Ensure that referential integrity is maintained between the two tables.

1. Create DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

2. Create STUDENT Table

```
CREATE TABLE STUDENT (
    SID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    DeptID INT,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 17),
    Status VARCHAR(20) DEFAULT 'Active',
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

3. Insert Sample Data

INSERT INTO DEPARTMENT

VALUES (101, 'Computer Science');

INSERT INTO STUDENT

(SID, Name, DOB, DeptID, Email, Age)

VALUES

(1, 'Arun', '2005-05-10', 101,

'arun@gmail.com', 20);

4. Display the Tables

SELECT * FROM DEPARTMENT;

SELECT * FROM STUDENT;

```
mysql> CREATE DATABASE UniversityDB2;
Query OK, 1 row affected (0.01 sec)

mysql> USE UniversityDB2;
Database changed
mysql> CREATE TABLE DEPARTMENT2 (
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(50) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE STUDENT2 (
    ->     SID INT PRIMARY KEY,
    ->     Name VARCHAR(50) NOT NULL,
    ->     DOB DATE NOT NULL,
    ->     DeptID INT,
    ->     Email VARCHAR(100) UNIQUE,
    ->     Age INT CHECK (Age >= 17),
    ->     Status VARCHAR(20) DEFAULT 'Active',
    ->     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT2(DeptID)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO DEPARTMENT2 VALUES
    -> (101, 'Computer Science'),
    -> (102, 'Information Technology');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO STUDENT2
    -> (SID, Name, DOB, DeptID, Email, Age)
    -> VALUES
    -> (1, 'Arun', '2005-05-10', 101, 'arun@gmail.com', 20),
    -> (2, 'Priya', '2006-03-15', 102, 'priya@gmail.com', 19);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM DEPARTMENT2;
+--------+------------------------+
| DeptID | DeptName               |
+--------+------------------------+
|    101 | Computer Science       |
|    102 | Information Technology |
+--------+------------------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM STUDENT2;
+-----+-------+------------+--------+-----------------+------+--------+
| SID | Name  | DOB        | DeptID | Email           | Age  | Status |
+-----+-------+------------+--------+-----------------+------+--------+
|   1 | Arun  | 2005-05-10 |    101 | arun@gmail.com  |   20 | Active |
|   2 | Priya | 2006-03-15 |    102 | priya@gmail.com |   19 | Active |
+-----+-------+------------+--------+-----------------+------+--------+
2 rows in set (0.00 sec)

mysql> DESC DEPARTMENT2;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptID   | int         | NO   | PRI | NULL    |       |
| DeptName | varchar(50) | NO   | UNI | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

```
mysql> DESC STUDENT2;
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| SID    | int          | NO   | PRI | NULL    |       |
| Name   | varchar(50)  | NO   |     | NULL    |       |
| DOB    | date         | NO   |     | NULL    |       |
| DeptID | int          | YES  | MUL | NULL    |       |
| Email  | varchar(100) | YES  | UNI | NULL    |       |
| Age    | int          | YES  |     | NULL    |       |
| Status | varchar(20)  | YES  |     | Active  |       |
+--------+--------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

Result

The tables are created successfully using PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. The foreign key maintains referential integrity between the `STUDENT` and `DEPARTMENT` tables.

Question:3

A retail company maintains a SALES(Sale_ID, Product_ID, Customer_ID, Quantity, Unit_Price, Sale_Date) relation. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions such as COUNT, SUM, AVG, MAX, and MIN to generate product-wise sales summaries, identify high-value products, and retrieve groups satisfying specified conditions.

Given Relation: `SALES(Sale_ID, Product_ID, Customer_ID, Quantity, Unit_Price, Sale_Date)`

1. COUNT – Count Total Sales per Product

```
SELECT Product_ID, COUNT(*) AS Total_Sales
FROM SALES
GROUP BY Product_ID;
```

2. SUM – Calculate Total Sales Amount per Product

```
SELECT Product_ID,
    SUM(Quantity * Unit_Price) AS Total_Amount
FROM SALES
GROUP BY Product_ID;
```

3. AVG – Calculate Average Unit Price per Product

```
SELECT Product_ID, AVG(Unit_Price) AS Average_Price
FROM SALES
GROUP BY Product_ID;
```

4. MAX – Find the Highest Unit Price per Product

```
SELECT Product_ID, MAX(Unit_Price) AS Highest_Price
FROM SALES
GROUP BY Product_ID;
```

5. MIN – Find the Lowest Unit Price per Product

```
SELECT Product_ID, MIN(Unit_Price) AS Lowest_Price
FROM SALES
GROUP BY Product_ID;
```

6. HAVING – Find Products with Total Sales Above ₹10,000

```
SELECT Product_ID,
    SUM(Quantity * Unit_Price) AS Total_Amount
FROM SALES
GROUP BY Product_ID
HAVING SUM(Quantity * Unit_Price) > 10000;
```

7. HAVING with COUNT – Products Having More Than 5 Sales

SELECT Product_ID, COUNT(*) AS Total_Sales
FROM SALES
GROUP BY Product_ID
HAVING COUNT(*) > 5;

```
mysql> CREATE DATABASE RetailDB2;
Query OK, 1 row affected (0.01 sec)

mysql> USE RetailDB2;
Database changed
mysql> CREATE TABLE SALES (
    ->     Sale_ID INT PRIMARY KEY,
    ->     Product_ID INT,
    ->     Customer_ID INT,
    ->     Quantity INT,
    ->     Unit_Price DECIMAL(10,2),
    ->     Sale_Date DATE
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO SALES VALUES
    -> (1, 101, 201, 2, 500, '2026-01-10'),
    -> (2, 102, 202, 3, 1000, '2026-01-11'),
    -> (3, 101, 203, 4, 500, '2026-01-12'),
    -> (4, 103, 204, 2, 2000, '2026-01-13'),
    -> (5, 102, 205, 1, 1000, '2026-01-14');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT Product_ID, COUNT(*) AS Total_Sales
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+-------------+
| Product_ID | Total_Sales |
+------------+-------------+
|        101 |           2 |
|        102 |           2 |
|        103 |           1 |
+------------+-------------+
3 rows in set (0.02 sec)
```

```
mysql> SELECT Product_ID,
    ->        SUM(Quantity * Unit_Price) AS Total_Amount
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+--------------+
| Product_ID | Total_Amount |
+------------+--------------+
|        101 |      3000.00 |
|        102 |      4000.00 |
|        103 |      4000.00 |
+------------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT Product_ID,
    ->        SUM(Quantity * Unit_Price) AS Total_Amount
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+--------------+
| Product_ID | Total_Amount |
+------------+--------------+
|        101 |      3000.00 |
|        102 |      4000.00 |
|        103 |      4000.00 |
+------------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT Product_ID, AVG(Unit_Price) AS Average_Price
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+---------------+
| Product_ID | Average_Price |
+------------+---------------+
|        101 |    500.000000 |
|        102 |   1000.000000 |
|        103 |   2000.000000 |
+------------+---------------+
3 rows in set (0.01 sec)
```

```
mysql> SELECT Product_ID, MAX(Unit_Price) AS Highest_Price
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+---------------+
| Product_ID | Highest_Price |
+------------+---------------+
|        101 |        500.00 |
|        102 |       1000.00 |
|        103 |       2000.00 |
+------------+---------------+
3 rows in set (0.00 sec)

mysql> SELECT Product_ID, MIN(Unit_Price) AS Lowest_Price
    -> FROM SALES
    -> GROUP BY Product_ID;
+------------+--------------+
| Product_ID | Lowest_Price |
+------------+--------------+
|        101 |       500.00 |
|        102 |      1000.00 |
|        103 |      2000.00 |
+------------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT Product_ID,
    ->        SUM(Quantity * Unit_Price) AS Total_Amount
    -> FROM SALES
    -> GROUP BY Product_ID
    -> HAVING SUM(Quantity * Unit_Price) > 2000;
+------------+--------------+
| Product_ID | Total_Amount |
+------------+--------------+
|        101 |      3000.00 |
|        102 |      4000.00 |
|        103 |      4000.00 |
+------------+--------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT Product_ID, COUNT(*) AS Total_Sales
    -> FROM SALES
    -> GROUP BY Product_ID
    -> HAVING COUNT(*) > 1;
+------------+-------------+
| Product_ID | Total_Sales |
+------------+-------------+
|        101 |           2 |
|        102 |           2 |
+------------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM SALES;
+---------+------------+-------------+----------+------------+------------+
| Sale_ID | Product_ID | Customer_ID | Quantity | Unit_Price | Sale_Date  |
+---------+------------+-------------+----------+------------+------------+
|       1 |        101 |         201 |        2 |     500.00 | 2026-01-10 |
|       2 |        102 |         202 |        3 |    1000.00 | 2026-01-11 |
|       3 |        101 |         203 |        4 |     500.00 | 2026-01-12 |
|       4 |        103 |         204 |        2 |    2000.00 | 2026-01-13 |
|       5 |        102 |         205 |        1 |    1000.00 | 2026-01-14 |
+---------+------------+-------------+----------+------------+------------+
5 rows in set (0.00 sec)
```

Result

The SQL queries generate product-wise sales summaries using `COUNT()`, `SUM()`, `AVG()`, `MAX()`, and `MIN()`. The `GROUP BY` clause groups records by product, while `HAVING` filters groups based on specified conditions.

Question:4

A company maintains EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID) records. Write a correlated subquery to identify employees whose salary is greater than the average salary of other employees working in the same department. Explain how the inner query is related to the outer query for each employee.

1. Create EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(50),
    Salary DECIMAL(10,2),
    Dept_ID INT
);
```

2. Insert Sample Records

```
INSERT INTO EMPLOYEE VALUES
(1, 'Arun', 40000, 10),
(2, 'Bala', 50000, 10),
(3, 'Kumar', 60000, 10),
(4, 'Priya', 30000, 20),
(5, 'Divya', 45000, 20);
```

3. Correlated Subquery

```
SELECT E.Emp_ID, E.Emp_Name, E.Salary, E.Dept_ID
FROM EMPLOYEE E
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
```

```
    FROM EMPLOYEE E2
    WHERE E2.Dept_ID = E.Dept_ID
      AND E2.Emp_ID <> E.Emp_ID
);
```

4. Display the Result

The query returns employees whose salary is greater than the average salary of the other employees in their department.

Emp_ID	Emp_Name	Salary	Dept_ID
2	Bala	50000	10
3	Kumar	60000	10
5	Divya	45000	20

5. Explanation

1. The outer query selects each employee.
2. The inner query calculates the average salary of other employees in the same department.
3. `E2.Dept_ID = E.Dept_ID` connects the inner query to the current employee's department.
4. `E2.Emp_ID <> E.Emp_ID` excludes the current employee from the average.
5. The `WHERE` condition displays employees earning more than that average.

```
mysql> CREATE DATABASE EmployeeDB2;
Query OK, 1 row affected (0.02 sec)

mysql> USE EmployeeDB2;
Database changed
mysql> CREATE TABLE EMPLOYEE (
    ->     Emp_ID INT PRIMARY KEY,
    ->     Emp_Name VARCHAR(50),
    ->     Salary DECIMAL(10,2),
    ->     Dept_ID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (1, 'Arun', 30000, 10),
    -> (2, 'Bala', 40000, 10),
    -> (3, 'Kumar', 50000, 10),
    -> (4, 'Priya', 25000, 20),
    -> (5, 'Divya', 45000, 20);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|      1 | Arun     | 30000.00 |      10 |
|      2 | Bala     | 40000.00 |      10 |
|      3 | Kumar    | 50000.00 |      10 |
|      4 | Priya    | 25000.00 |      20 |
|      5 | Divya    | 45000.00 |      20 |
+--------+----------+----------+---------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT E.Emp_ID, E.Emp_Name, E.Salary, E.Dept_ID
    -> FROM EMPLOYEE E
    -> WHERE E.Salary > (
    ->     SELECT AVG(E2.Salary)
    ->     FROM EMPLOYEE E2
    ->     WHERE E2.Dept_ID = E.Dept_ID
    ->       AND E2.Emp_ID <> E.Emp_ID
    -> );
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|      3 | Kumar    | 50000.00 |      10 |
|      5 | Divya    | 45000.00 |      20 |
+--------+----------+----------+---------+
2 rows in set (0.00 sec)
```

Result: The correlated subquery identifies employees earning more than their other department colleagues' average salary.

Question:5

A software company maintains EMPLOYEE(Emp_ID, Emp_Name, Dept_ID) and PROJECT(Project_ID, Project_Name, Emp_ID) relations. Apply INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, and CROSS JOIN to retrieve employee–project information. Explain the type of result produced by each join operation.

Given Relations:

- `EMPLOYEE(Emp_ID, Emp_Name, Dept_ID)`
- `PROJECT(Project_ID, Project_Name, Emp_ID)`

1. INNER JOIN

Displays employees who are assigned to projects.

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
INNER JOIN PROJECT P
ON E.Emp_ID = P.Emp_ID;
```

Result: Returns only matching employee and project records.

2. LEFT OUTER JOIN

Displays all employees, including those not assigned to projects.

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
LEFT JOIN PROJECT P
ON E.Emp_ID = P.Emp_ID;
```

Result: Returns all employees; unmatched project details are `NULL`.

3. RIGHT OUTER JOIN

Displays all projects, including those not assigned to an employee.

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
RIGHT JOIN PROJECT P
ON E.Emp_ID = P.Emp_ID;
```

Result: Returns all projects; unmatched employee details are `NULL`.

4. FULL OUTER JOIN

Displays all employees and all projects, including unmatched records.

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
LEFT JOIN PROJECT P
ON E.Emp_ID = P.Emp_ID

UNION

SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
RIGHT JOIN PROJECT P
ON E.Emp_ID = P.Emp_ID;
```

Result: Returns matching and non-matching records from both tables. MySQL 8.0 does not directly support `FULL OUTER JOIN`, so `UNION` combines the left and right joins.

5. CROSS JOIN

Combines every employee with every project.

```
SELECT E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
CROSS JOIN PROJECT P;
```

```
mysql> CREATE DATABASE ProjectDB2;
Query OK, 1 row affected (0.02 sec)

mysql> USE ProjectDB2;
Database changed
mysql> CREATE TABLE EMPLOYEE (
    ->     Emp_ID INT PRIMARY KEY,
    ->     Emp_Name VARCHAR(50),
    ->     Dept_ID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE PROJECT (
    ->     Project_ID INT PRIMARY KEY,
    ->     Project_Name VARCHAR(50),
    ->     Emp_ID INT
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (1, 'Arun', 10),
    -> (2, 'Priya', 20),
    -> (3, 'Ravi', 10);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO PROJECT VALUES
    -> (101, 'Website', 1),
    -> (102, 'Database', 2),
    -> (103, 'Mobile App', NULL);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

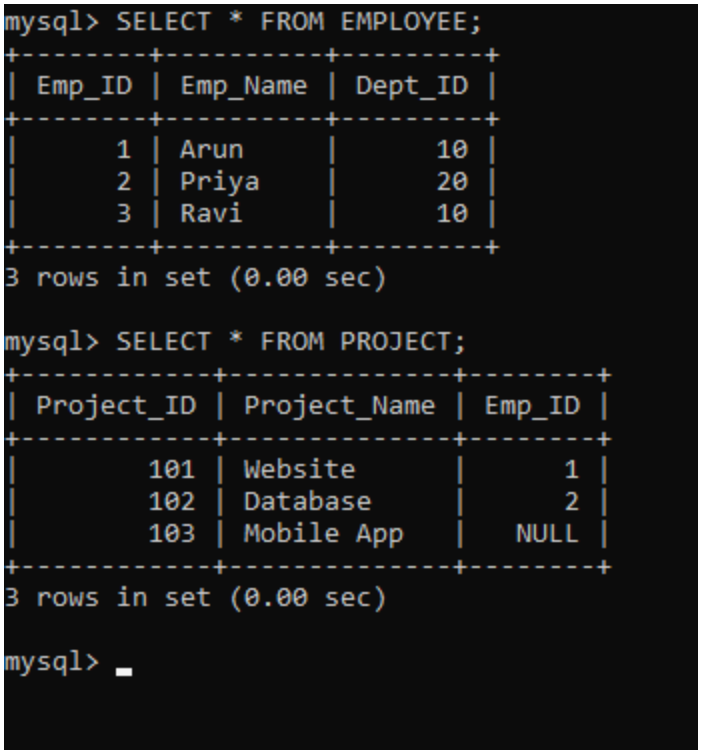
```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> INNER JOIN PROJECT P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+--------------+
| Emp_ID | Emp_Name | Project_Name |
+--------+----------+--------------+
|      1 | Arun     | Website      |
|      2 | Priya    | Database     |
+--------+----------+--------------+
2 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> LEFT JOIN PROJECT P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+--------------+
| Emp_ID | Emp_Name | Project_Name |
+--------+----------+--------------+
|      1 | Arun     | Website      |
|      2 | Priya    | Database     |
|      3 | Ravi     | NULL         |
+--------+----------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> RIGHT JOIN PROJECT P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+--------------+
| Emp_ID | Emp_Name | Project_Name |
+--------+----------+--------------+
|      1 | Arun     | Website      |
|      2 | Priya    | Database     |
|   NULL | NULL     | Mobile App   |
+--------+----------+--------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> LEFT JOIN PROJECT P
    -> ON E.Emp_ID = P.Emp_ID
    ->
    -> UNION
    ->
    -> SELECT E.Emp_ID, E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> RIGHT JOIN PROJECT P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+--------------+
| Emp_ID | Emp_Name | Project_Name |
+--------+----------+--------------+
|      1 | Arun     | Website      |
|      2 | Priya    | Database     |
|      3 | Ravi     | NULL         |
|   NULL | NULL     | Mobile App   |
+--------+----------+--------------+
4 rows in set (0.00 sec)

mysql> SELECT E.Emp_Name, P.Project_Name
    -> FROM EMPLOYEE E
    -> CROSS JOIN PROJECT P;
+----------+--------------+
| Emp_Name | Project_Name |
+----------+--------------+
| Ravi     | Website      |
| Priya    | Website      |
| Arun     | Website      |
| Ravi     | Database     |
| Priya    | Database     |
| Arun     | Database     |
| Ravi     | Mobile App   |
| Priya    | Mobile App   |
| Arun     | Mobile App   |
+----------+--------------+
9 rows in set (0.00 sec)
```



Result: Returns every possible employee–project combination.

Conclusion

SQL joins combine employee and project data in different ways. `INNER JOIN` returns matches, outer joins preserve unmatched records, and `CROSS JOIN` returns all possible combinations.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation